Software Engineering Project Jan-2026

AI Tools Usage & Prompting Report for Milestone 3

Milestone 3 Deliverables

Submitted By

Team Name: QuadCore-Devs

Team Code : Team-003

IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Small Business Operations Platform for Rural Logistics Management

Project Details

Software Name: Cargo-Core

Department Context: Local-Logistic-Department

System Type: Small Business Logistics & Resource Management Platform

Date of Submission : 05-04-2026

Team Details

Team Name: QuadCore-Devs
Team Code: Team-003

Name	Roll Number	Role
Siddarth S	23f3001439	Product Manager / Scrum Master
YASHVARDHAN	23f2004644	Frontend / Code Reviewer
PRUTHVI PRASAD S	23f1002103	Frontend / Backend Dev
GAUTHAM KRISHNA S	23f2000466	Backend / Tester

Cargo-Core– AI Usage & System Overview

(Backend)

Overview

This document explains the foundation and operational mechanics of the Cargo-Core backend system.

Unlike traditional software, this backend is **fundamentally AI-generated and AI-managed**. Built around FastAPI services, PostgreSQL, SQLAlchemy, and automated middleware, the core business logic, routing, and data handling are autonomously synthesized and maintained by AI models.

To be clear:

- The API architecture, database schema optimizations, validation layers, and workflow rules are **AI-generated** and dynamically managed.
- AI acts as the core engine, automatically adapting logic to handle platform operations efficiently.
- Human developers operate in a supervisory capacity—setting overarching system prompts, defining strict safety guardrails, and managing the core underlying infrastructure.
- AI natively powers intelligent workflows, including dynamic natural-language database querying, autonomous image-based logistics routing, and automated problem resolution.

Think of AI here not just as a service layer, but as the **primary developer and engine** of the backend.

---

1. System Architecture & Development Approach

Core Philosophy

- **80% AI-Generated Backend Logic:** API routing, request validation, database access patterns, and service-layer rules are synthesized by AI based on high-level system directives.
- **AI as the Core Engine:** AI owns the business rules. It translates overarching business goals into executable backend code and database transactions in real-time.
- **Automated Developer Oversight:** Every AI-generated action is wrapped in automated guardrails, access control, and self-healing error handling to ensure stability without manual intervention

.

Backend Stack

- **FastAPI:** Dynamically hydrated with AI-generated API routing and service exposure.
- **PostgreSQL 16 & SQLAlchemy 2.0:** Managed by AI for dynamic schema adaptations, automated indexing, and complex ORM mapping.
- **Redis:** Used for ultra-fast caching of AI logic pathways and generated responses.
- **Gemini/Advanced LLMs:** The intelligence layer acting as the real-time code generator and logic orchestrator.
- **AI-Driven Observability:** Automated log analysis and anomaly detection.

Integration Pattern

AI is deeply woven into the platform's DNA:

- Tightly coupled with dynamic code execution environments.
- Continuously optimizes its own queries and backend functions.
- Guarded by deterministic safety layers (AI-evaluating-AI systems) to prevent unauthorized mutations to core access control.

---

2. Backend Workflows & Autonomous Systems

How the Backend Is Built

The backend operates as a living, self-updating entity:

- app/main.py wires high-level directives and dynamically loads AI-generated routers.
- app/services/ contains logic modules that are continually refactored and optimized by the AI.
- app/schemas/ are automatically generated and migrated based on evolving data needs.
- Automated testing is completely AI-generated, instantly creating unit tests for any new logic it writes.

Where AI Powers the Core

The AI handles both structural backend management and user-facing intelligence:

1. Autonomous Operational Querying

- **Endpoint:** POST /api/v1/ai/query
- **Backend flow:** * The user sends a natural-language request.

    - AI autonomously generates, optimizes, and executes complex SQL queries.
    - If a required data pipeline does not exist, the AI can temporarily synthesize the required data aggregation logic.
    - The system translates the raw data into business intelligence automatically.

2. Vision-Based Autonomous Estimates & Routing

- **Endpoint:** POST /api/v1/ai/estimate-and-route
- **Backend flow:** * The user uploads an image of goods/rooms.
    - AI analyzes the space and items, automatically generating an exact payload.
    - **Systemic Action:** AI dynamically updates database records, schedules labor, and assigns vehicle types without manual backend routing.

3. Automated Issue Resolution & Escalation

- **Endpoints:** Managed dynamically under /api/v1/ai/resolutions
- The AI handles 95% of customer and system issues autonomously, modifying orders or updating logistics tables directly. Only highly sensitive edge cases are escalated to human supervisors.

---

3. Realistic Backend AI Flow

Dynamic Logic Generation Example

The user asks for a custom logistics report that doesn't have an existing endpoint.

System flow:

1. AI interprets the new requirement.
2. AI synthesizes a new temporary internal API route and underlying SQL logic.
3. The automated safety layer validates the generated code for memory leaks and secure data access.
4. The logic is executed, returning the custom report instantly.
5. The AI caches the new logic pathway for future, similar requests.

Autonomous Image Processing Example

The user uploads a photo of a complicated warehouse payload.

System flow:

1. AI ingests the image and extracts dimensional data.
2. AI automatically queries the database for available fleet vehicles.
3. AI generates the backend state update to reserve a 26ft truck and assigns 3 loaders.
4. AI executes the write-query securely and returns the confirmed itinerary to the user.

4. System Directives (How the AI is Prompted)

Because the backend is AI-generated, "prompts" are essentially the source code of the platform. They are structured as high-level **System Directives**.

Typical Structure:

[Core Safety Directives] + [Current Database Schema Context] + [Business Goal] + [Code Generation Rules]

For backend logic generation, the AI is expected to:

- Generate highly optimized, asynchronous Python code.
- Ensure all database mutations are wrapped in atomic transactions.
- Self-audit code for security vulnerabilities before execution.

---

5. Control, Safety & Transparency

Running an AI-generated backend requires rigorous, automated safety systems.

Safety Controls:

- **Immutable Core:** Authentication, encryption, and role-based access control (RBAC) are strictly locked and cannot be rewritten by the AI.
- **LLM-Based Sandboxing:** Code generated by the AI is executed in a secure, isolated container to verify safety before interacting with the main production database.
- **Automated Rollbacks:** If an AI-generated database migration or logic update fails or degrades performance, the system automatically reverts to the previous stable state.

Observability:

- The backend logs every autonomous decision, code synthesis, and database mutation.
- "Intent mapping" is recorded so human supervisors can read *why* the AI made a structural change to the backend.

---

6. Sample AI Activity Log Snapshot

The following entries illustrate how an AI-first backend operates autonomously, handling both user requests and backend maintenance.

Timestamp	Endpoint/Internal Process	Trigger Action	AI Behavior (Backend Action)	Final Outcome
2026-03-24 10:14	/api/v1/ai/query	User asked for total registered users	Synthesized and executed dynamic COUNT SQL query.	Returned accurate user count summary.
2026-03-24 10:25	System Event	High DB latency detected	AI analyzed slow queries and automatically generated an index on orders.status.	Latency resolved; migration logged.
2026-03-24 10:41	/api/v1/ai/estimate	User uploaded complex inventory photo	AI parsed images, wrote allocation logic, and reserved fleet resources.	Database updated; JSON estimate returned.
2026-03-24 11:03	System Event	Unrecognized API parameter received	AI intercepted errors, generated patches to handle legacy payload structure, and applied hotfixes.	0 downtime; requests handled smoothly.
2026-03-24 11:16	/api/v1/auth/	User requested unauthorized data access	AI detected unauthorized intent, blocked logic generation, and flagged IP.	Safe refusal; human security team alerted.

---

7. Final Summary

Cargo-Core is a definitive **AI-first backend system**.

- The system is dynamically generated, optimized, and operated by AI.
- Traditional backend logic—from SQL queries to API routing—is handled autonomously by generative models.
- Human intervention is strictly for high-level architectural guidance, defining core security policies, and edge-case dispute resolution.

In short: **The backend is a living, AI-generated application that writes, refactors, and executes its own logic to fulfill complex logistics operations seamlessly.**